

Google CTF 2022 Crypto - Maybe Someday

deuterium — 2022-07-04

Canonical HTML: <https://deut-erium.github.io/WriteUps/2022/google-ctf/crypto/maybe-someday/2022-07-04-Google-CTF-22-Maybe-Someday.html>

Abstract

Google CTF 2022 cryptography writeup maybe someday paillier cryptosystem
homomorphic padding oracle

TLDR; Leaking 20 bits of information from custom padding oracle using homomorphic properties of paillier cryptosystem

Challenge Description

Maybe Someday [240 points] (solved by 35)

Leave me your ciphertexts. I will talk to you later.
maybe-someday.2022.ctfcompetition.com 1337
[attachment](#)

Files

[attachment.zip](#)

- [chall.py](#)

Not Included

[/flag.txt](#)

Server Source

```
from Crypto.Util.number import getPrime as get_prime
import math
import random
import os
import hashlib

# Suppose gcd(p, q) = 1. Find x such that
# 1. 0 <= x < p * q, and
# 2. x = a (mod p), and
# 3. x = b (mod q).
def crt(a, b, p, q):
    return (a*pow(q, -1, p)*q + b*pow(p, -1, q)*p) % (p*q)

def L(x, n):
    return (x-1) // n
```

```

class Paillier:
    def __init__(self):
        p = get_prime(1024)
        q = get_prime(1024)

        n = p * q
        λ = (p-1) * (q-1) // math.gcd(p-1, q-1) # lcm(p-1, q-1)
        g = random.randint(0, n-1)
        μ = pow(L(pow(g, λ, n**2), n), -1, n)

        self.n = n
        self.λ = λ
        self.g = g
        self.μ = μ

        self.p = p
        self.q = q

# https://www.rfc-editor.org/rfc/rfc3447#section-7.2.1
def pad(self, m):
    padding_size = 2048//8 - 3 - len(m)

    if padding_size < 8:
        raise Exception('message too long')

    random_padding = b'\0' * padding_size
    while b'\0' in random_padding:
        random_padding = os.urandom(padding_size)

    return b'\x00\x02' + random_padding + b'\x00' + m

def unpad(self, m):
    if m[:2] != b'\x00\x02':
        raise Exception('decryption error')

    random_padding, m = m[2:].split(b'\x00', 1)

    if len(random_padding) < 8:
        raise Exception('decryption error')

    return m

def public_key(self):
    return (self.n, self.g)

def secret_key(self):
    return (self.λ, self.μ)

def encrypt(self, m):
    g = self.g
    n = self.n

    m = self.pad(m)
    m = int.from_bytes(m, 'big')

    r = random.randint(0, n-1)
    c = pow(g, m, n**2) * pow(r, n, n**2) % n**2

    return c

```

```

def decrypt(self, c):
    λ = self.λ
    μ = self.μ
    n = self.n

    m = L(pow(c, λ, n**2), n) * μ % n
    m = m.to_bytes(2048//8, 'big')

    return self.unpad(m)

def fast_decrypt(self, c):
    λ = self.λ
    μ = self.μ
    n = self.n
    p = self.p
    q = self.q

    rp = pow(c, λ, p**2)
    rq = pow(c, λ, q**2)
    r = crt(rp, rq, p**2, q**2)
    m = L(r, n) * μ % n
    m = m.to_bytes(2048//8, 'big')

    return self.unpad(m)

def challenge(p):
    secret = os.urandom(2)
    secret = hashlib.sha512(secret).hexdigest().encode()

    c0 = p.encrypt(secret)
    print(f'{c0 = }')

    # # The secret has 16 bits of entropy.
    # # Hence 16 oracle calls should be sufficient, isn't it?
    # for _ in range(16):
    #     c = int(input())
    #     try:
    #         p.decrypt(c)
    #         print('😊')
    #     except:
    #         print('😡')

    # I decided to make it non-interactive to make this harder.
    # Good news: I'll give you 25% more oracle calls to compensate, anyways.
    cs = [int(input()) for _ in range(20)]
    for c in cs:
        try:
            p.fast_decrypt(c)
            print('😊')
        except:
            print('😡')

    guess = input().encode()

    if guess != secret: raise Exception('incorrect guess!')

def main():
    with open('flag.txt', 'r') as f:
        flag = f.read()

```

```

p = Paillier()
n, g = p.public_key()
print(f'{n = }')
print(f'{g = }')

try:
    # Once is happenstance. Twice is coincidence...
    # Sixteen times is a recovery of the pseudorandom number generator.
    for _ in range(16):
        challenge(p)
        print('💡')
    print(f'🚩 {flag}')
except:
    print('👋')

if __name__ == '__main__':
    main()

```

Understanding the challenge

- The challenge server deploys the Paillier Cryptosystem
- A single instance `p` of `Paillier` is used through the connection and public key `(n, g)` is provided
- We are required to solve `challenge(p)` 16 times successfully to get the flag

challenge(p)

- A two-byte secret is selected randomly which is then hashed to its sha512 hexdigest byte-string of length 128 hexadecimal characters
 - if selected secret = `b'\x00\x00'`,
 - `hashlib.sha512(secret).hexdigest().encode() =`
`b'5ea71dc6d0b4f57bf39aadd07c208c35f06cd2bac5fde210397f70de11d439c62ec1cdf3183758865fd387fcea0bada2f6c37a4a17851dd1d78fefe6f204ee54'`
 - 20 integer inputs taken together after which the server returns 20 outputs whether decryption succeeded for each of the inputs '😊' for success and '😞' for failure
 - After 20 inputs, it requests for a guess for secret if the guess matches, we are good to go

Understanding Decryption

- `fast_decrypt(c)` is same as `decrypt(c)`. The only difference being the way $r = \text{pow}(c, \lambda, n^{**2})$ is calculated using Chinese Remainder Theorem (just for speed considerations and no effect on the workings of the challenge)
- All steps will proceed in decryption of arbitrary integer `c`, just the last `self.unpad(m)` which will fail if `m` is not in desired padding format.

Understanding Padding/Unpadding

Valid Padding Structure

```
+-----+-----+-----+-----+
| \x00\x02 | RANDOM_NONNULL_DATA | \x00 | message |
+-----+-----+-----+-----+
```

Where

- The first two bytes are 0 and 2 respectively
- RANDOM_NONNULL_DATA is a random string constructed from bytes [1,255] of length $2048//8 - 3$ (for `\x00\x02` and `\x00`) - $\text{len}(\text{message}) = 253 - \text{len}(\text{message}) \div 8$ i.e. from [8,253] (funnily enough message can be null)
- A null byte separating RANDOM_NONNULL_DATA from our message
- Our message of length [0,245] from restriction on size of padding

While unpadding, the server just checks if these conditions are satisfied and raises exception otherwise.

Homomorphic properties of Paillier cryptosystem

RSA padding oracle attack?

Cryptopals RSA padding oracle challenge

Solution Ideas

To solve the challenge, we would like to build a differentiator which would cut the secret space in half on each query (ideally).

i.e We can create a ciphertext c_1 from c_0 utilizing multiplications and additions (or subtractions)

Idea 1

Adding a number to plaintext

We can differentiate numbers utilizing the addition property, i.e if we add a big enough number, the resulting decrypted number $\geq 2^{1024}$ thus overflowing the null padding byte and resulting in invalid decryption.

We can effectively cut the secret space to half using this. Sorting the secrets, picking the middle secret s_{32768} and finding the value v such that $s_{32768} + v \geq 2^{1024}$ which will fail for all numbers $\geq s_{32768}$ and pass otherwise.

This would enable a binary search over the possible secrets and we would be able to get the secret in 16 queries!

BUT

That requires us to adapt our queries based on the outcomes of previous queries. Here we are given 20 queries and the results are provided all together.

Idea 2

Making the default as failure and forming `\x00` inside the message

i.e filling the padding byte by adding $[1,255]*2^{**}1024$ to the message and subtracting the first '0' byte i.e $48*2^{**}1016$ from it.

This way all the messages having the byte '0' as the MSB, would result in valid decryption and all others would fail.

```
+-----+-----+-----+-----+-----+-----+
| \x00\x02 | RANDOM_NONNULL_DATA | NON_NULL_BYTE | message1 | \x00 | message2 |
+-----+-----+-----+-----+-----+-----+
```

Example

suppose the message is

5ea71dc6d0b4f57bf39aadd07c208c35f06cd2bac5fde210397f70de11d439c62ec1cdf3183758865fd387fc
ea0bada2f6c37a4a17851dd1d78fefe6f204ee54

Then the padded message would look like

```
+-----+-----+-----+-----+
| \x00\x02 | RANDOM_NONNULL_DATA | \x00 | 5ea71.. |
+-----+-----+-----+-----+
```

b'\x00\x02XX...XXXX\x005YY...YY'

where `RANDOM_NONNULL_DATA` is placeholder for $256-3-128 = 125$ random non-null bytes

adding $2^{**}1024$ to the message it would look something like

```
+-----+-----+-----+-----+
| \x00\x02 | RANDOM_NONNULL_DATA | \x01 | 5ea71.. |
+-----+-----+-----+-----+
```

Resulting in no null bytes, hence an invalid message

Now subtracting `ord("5")*2**1024` from the message we would get something like,

```
+-----+-----+-----+-----+
| \x00\x02 | RANDOM_NONNULL_DATA | \x01 | \x00 | ea71.. |
+-----+-----+-----+-----+
```

Which the server would believe to be a padding for the message

ea71dc6d0b4f57bf39aadd07c208c35f06cd2bac5fde210397f70de11d439c62ec1cdf3183758865fd387fce
a0bada2f6c37a4a17851dd1d78fefe6f204ee54 and $125+1 = 126$ random non-null bytes

Interesting Observation

If the first byte was anything less than the value 53, it would have carried back 1 from a previous byte thus making it `\x00` again hence making it a valid value again. This would help us differentiate half the values if we subtract `ord("8") = 56`.

Solve Scripts

The full attack is implemented in the attached scripts:

- [`solve.py`](#) — end-to-end solver: sha512 secret space, padding-oracle queries against the Paillier homomorphic blinding, and a decision-tree classifier (with simulated annealing) to pick maximally-distinguishing query positions
- [`build\_classifier.py`](#) — builds the classifier used to narrow candidate secrets
- [`chall.py`](#) — the challenge server for local testing

| Interactive controls are omitted from this PDF. See the canonical HTML version.